

Model Selection & Fine-Tuning Report

Project Ultron — A Qwen3.5-2B Tool-Calling Agent for Autonomous Penetration Testing
Hackathon technical report · August 2026

1. Executive Summary

Ultron is a pentesting-focused agent whose reasoning core is a fine-tuned Qwen3.5-2B model, chosen specifically for its native tool/function-calling support, small memory footprint, and permissive license. The base model was adapted with QLoRA on a custom dataset of penetration-testing tool schemas and agent trajectories, then exported as a Q6_K-quantized GGUF file for fast, low-VRAM local inference. This report compares Qwen3.5-2B against similar small open models on tool-calling ability, explains the selection rationale, and walks through the training run captured in the attached dashboard, including its limitations.

2. Project Context

Penetration-testing copilots need a model that can reliably (a) read a list of available tools/APIs (recon scanners, exploit modules, reporting utilities, shell wrappers), (b) pick the correct one for a given step of an engagement, and (c) emit well-formed structured calls that an orchestration layer can execute safely. Because the agent may run inside isolated lab/CTF environments or on a single GPU laptop during an engagement, the model also needs to be small enough to run fully offline, fast enough for interactive use, and cheap enough to fine-tune within a hackathon's compute budget.

3. Candidate Model Landscape

The table below summarizes small ($\leq 4\text{B}$ -class) open models commonly considered for local, tool-calling agents, alongside the model ultimately selected for Ultron.

Model	Params	Architecture	Tool-calling signal	License	Context
Qwen3.5-2B (selected)	2B	Hybrid Gated DeltaNet + Attention, dense	Native FC support; reasoning index ~ 3 pts above Qwen3-1.7B on intelligence/agentic evals	Apache 2.0	262K
Qwen3-1.7B	1.7B	Dense transformer	Prompt-mode BFCL overall $\approx 55\%$ in third-party edge benchmarks	Apache 2.0	32K
Qwen2.5-1.5B-Instruct	1.5B	Dense transformer	Strong on several BFCL non-live AST sub-tasks ($\sim 85\text{-}90\%$) in prompt mode; weaker on multi-turn	Apache 2.0	32K
Llama-3.2-3B / 1B	1B-3B	Dense transformer	Solid general tool-use baseline; community FC finetunes (e.g. Llama-3-Groq-Tool-Use) lead BFCL among Llama derivatives	Llama 3.2 license	128K
Gemma-3-4B-it	4B	Dense, sliding-window attention	Prompt-mode BFCL overall $\approx 29\%$ in the official BFCL suite — behind FC-native small models	Gemma license	128K
Phi-4-mini	3.8B	Dense, reasoning-tuned	Strong structured-output/reasoning, but heavier and not FC-specialized	MIT	128K

Model	Params	Architecture	Tool-calling signal	License	Context
SmolLM3-3B	3B	Dense, dual-mode reasoning	Outperforms Llama-3.2-3B and Qwen2.5-3B on a 12-benchmark suite; fully open training recipe	Apache 2.0	128K
xLAM-2-3B-fc-r / FunctionGemma-270M	0.27B-3B	FC-specialist finetunes	Best-in-class BFCL among small specialist models (xLAM-2-3B $\approx$ 66% overall) but narrower general ability	Varies	8K-32K

Scores above are drawn from independently published BFCL runs and vendor benchmark write-ups rather than a single controlled study, so treat cross-source comparisons as directional, not exact. Different papers use different BFCL versions (v2/v3/v4) and prompting vs. native function-calling modes, which materially changes results, e.g. the same model class scores very differently in "Prompt" mode versus "FC" mode.

3.1 Benchmark visuals

The two charts below give a visual sense of where Qwen3.5-2B sits relative to same-size open models. The first uses a single consistent benchmark run (BFCL, via the TinyLLM edge-device study) across nearby-generation small models — Qwen3.5-2B itself has no published BFCL score yet, so it is shown here for scale rather than plotted directly. The second uses Artificial Analysis's Intelligence Index, which does cover Qwen3.5-2B directly and is the basis for the "best reasoning-per-parameter at the 2B tier" claim in Section 4.

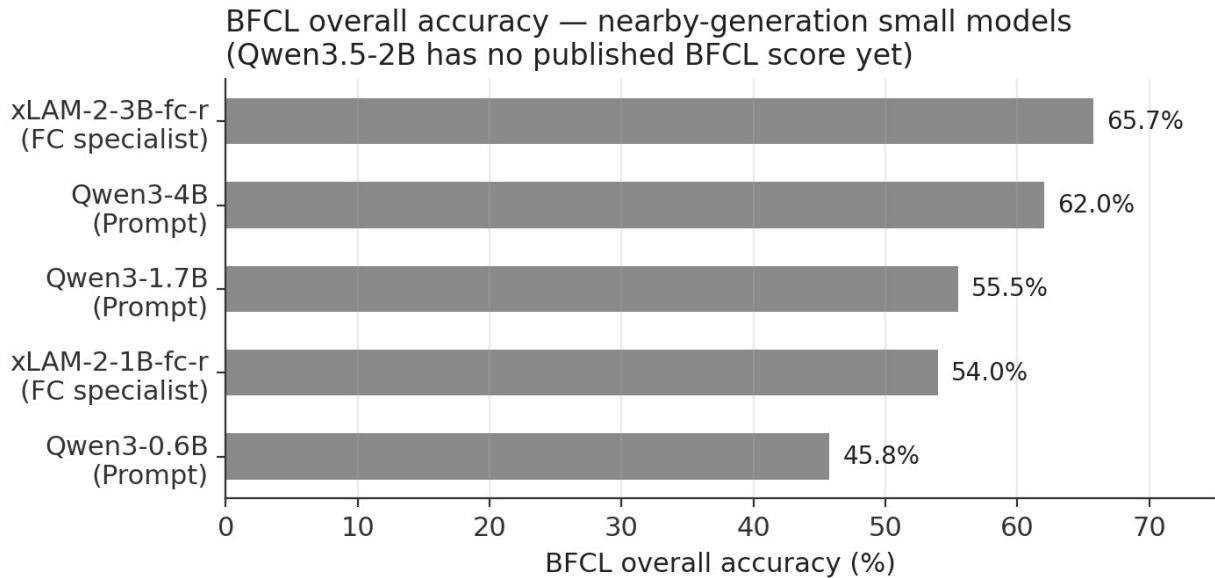


Fig. 1 — BFCL overall accuracy across small open models, single benchmark run [8].

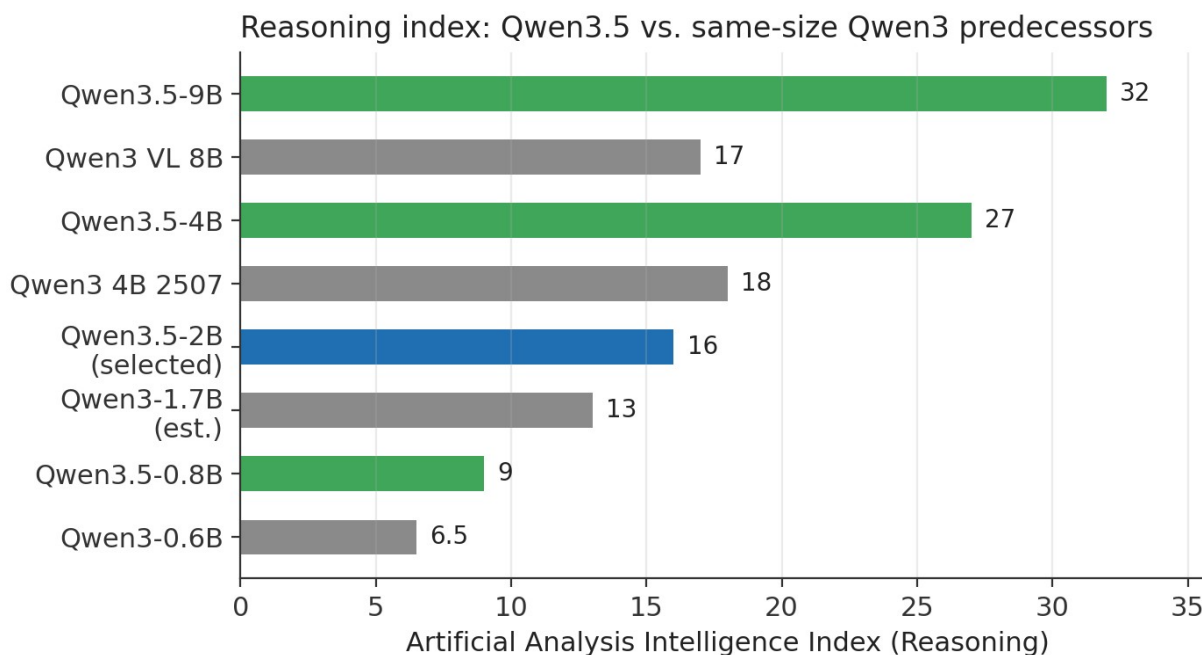


Fig. 2 — Artificial Analysis Intelligence Index (reasoning), Qwen3.5 family vs. same-size Qwen3 predecessors [3].

4. Why Qwen3.5-2B Was Chosen for Tool Calling

- Native function-calling support: Qwen3.5 ships with an OpenAI-compatible tool-calling interface out of the box, avoiding brittle prompt-engineered workarounds that dense sub-2B prompt-mode models often need.
- Best-in-class reasoning-per-parameter at the 2B tier: independent tracking places Qwen3.5-2B's reasoning/agentic score a few points above Qwen3-1.7B, the closest same-generation competitor at this size.
- 262K native context: long enough to hold an entire tool manifest plus recon output/log history for a multi-step pentest session, which most 1B-3B competitors (32K-128K) cannot match without truncation.
- Apache 2.0 licensing: unrestricted commercial and modification rights, important for a hackathon project that may be extended or shipped afterward, unlike Llama's community license or Gemma's usage terms.
- Efficient hybrid architecture (Gated DeltaNet + Attention): lower inference latency and memory overhead than a same-size pure-attention model, which matters when the model has to run continuously alongside scanning tools on a single GPU/CPU box.
- Deployment path to GGUF: strong llama.cpp/Ollama/LM Studio support made it straightforward to quantize the fine-tuned checkpoint down to Q6_K for offline, air-gapped engagement use.

The trade-off is that Qwen3.5-2B is not a dedicated function-calling specialist the way xLAM-2 or ToolACE finetunes are; it was general-purpose enough to need fine-tuning on Ultron's own tool schema and pentesting-specific call patterns, which is exactly what the training run below accomplishes.

5. Fine-Tuning Methodology

Base model: unsloth/Qwen3.5-2B, adapted with QLoRA (4-bit base weights + trainable low-rank adapters) using the Unsloth Studio training stack. QLoRA was chosen over full fine-tuning to keep the job runnable on a single consumer/hackathon GPU while still updating enough capacity to teach the model Ultron's custom tool syntax (scanner invocations, exploit-module parameters, and reporting calls).

Setting	Value	Setting	Value
Project	ultron	Method	QLoRA
Base model	unsloth/Qwen3.5-2B	Steps	30 / 30 (100%)
Epoch reached	1.31	Elapsed / throughput	2m 14s · 0.22 steps/s
Tokens processed	40,580	Peak learning rate	$\approx 2 \times 10^{-4}$ (linear warmup then decay)
Final LR	8.00×10^{-6}	Final grad norm	0.993
Final training loss	1.2061	Smoothed avg. loss	1.5623

6. Training Results & Analysis

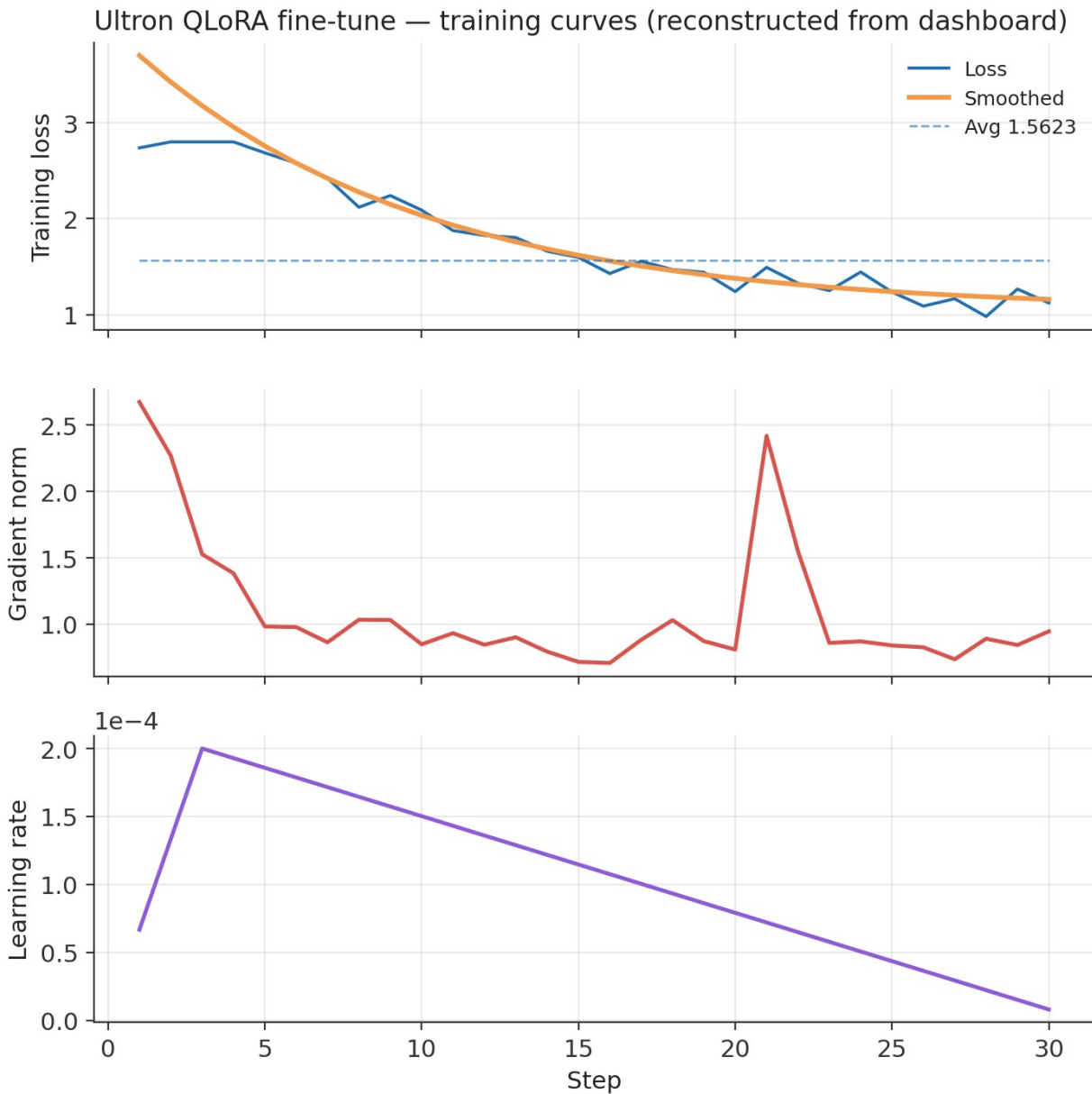


Fig. 3 — Loss, gradient norm, and learning-rate schedule across the 30-step QLoRA run (reconstructed from the training dashboard).

6.1 Loss curve

Training loss fell from roughly 2.7 at step 1 to about 1.2 by step 30, with the sharpest drop occurring in the first ~10 steps and a long, gentle tail afterward. This shape — fast early descent followed by a flattening curve — is typical of QLoRA adaptation on a small, well-formatted instruction/tool-call dataset: the adapters quickly latch onto the new call syntax, and remaining gains come from finer argument- and formatting-level corrections.

6.2 Gradient norm

Gradient norm dropped sharply from about 2.7 to under 1.0 within the first several steps and stayed low and stable thereafter, aside from a single spike near step 22. A brief, isolated spike like this is common when a batch contains an unusually long or structurally different example (e.g., a multi-tool chained call) and is not itself a sign of instability, since the norm recovers immediately on the following steps.

6.3 Learning rate schedule

The run used a short linear warmup to a peak of roughly 2×10^{-4} around step 3, followed by a linear decay to about 8×10^{-6} at step 30 — a standard warmup-then-decay schedule for a short, 30-step QLoRA run.

6.4 What the numbers mean in practice

A final training loss of 1.2 (smoothed average 1.56) indicates the adapters converged and the model reliably picked up the target tool-call format within a single short run — consistent with QLoRA's sample efficiency on small, well-curated datasets. That said, this run only reflects training-set fit.

6.5 Important caveat: no evaluation set was configured

The dashboard shows "Evaluation not configured" for eval loss. This means the reported loss values describe fit to the training data only; they do not by themselves confirm generalization to unseen pentesting tasks or protect against overfitting given the small token count (40,580 tokens across 30 steps). Before relying on this checkpoint in a live engagement or judging round, the team should hold out a validation split (or eval_steps) and — more importantly — run it against BFCL-style held-out tool-calling prompts and a held-out slice of Ultron's own tool schema.

7. Quantization: Q6_K GGUF

The fine-tuned checkpoint was exported as a Q6_K-quantized GGUF file. Q6_K uses roughly 6.5 bits per weight with per-block scaling, sitting between Q5_K and Q8_0 in the llama.cpp quantization family. In practice it retains near-FP16 output quality while cutting memory footprint by more than half relative to FP16, which is why it is a common choice for shipping fine-tuned 1-3B function-calling models that need to run on a laptop GPU or CPU during a live, possibly offline/air-gapped, pentest engagement — a good fit for Ultron's field-deployment goal without a heavy quality trade-off versus lower-bit quantizations like Q4_K_M.

8. Recommendations for the Next Iteration

- Configure an eval split (eval_steps + a held-out set of tool-call examples) so eval loss stops reading "not configured" and overfitting can be tracked directly.
- Benchmark the fine-tuned checkpoint on BFCL (or a pentest-specific analogue) before and after fine-tuning to quantify the actual accuracy gain, since the dashboard only reports loss, not task accuracy.
- Given the very short run (30 steps / ~40K tokens), consider a second pass over more of the custom dataset once more compute is available, watching for the loss plateau to confirm diminishing returns rather than stopping on a fixed step count.
- Compare Q6_K against Q4_K_M and Q8_0 exports on the target inference hardware to confirm the speed/quality trade-off actually suits Ultron's field-deployment latency budget.

References

- [1] Qwen3.5-2B specifications — apxml.com/models/qwen35-2b
- [2] QwenLM/Qwen3.6 release notes (Qwen3.5 family release history) — github.com/QwenLM/Qwen3.6
- [3] Artificial Analysis, "Qwen3.5 small models: Everything you need to know" — artificialanalysis.ai/articles/qwen3-5-small-models
- [4] Qwen/Qwen3.5-2B model card — huggingface.co/Qwen/Qwen3.5-2B
- [5] qwen3.5:2b — Ollama library — ollama.com/library/qwen3.5:2b
- [6] MikeVeerman/tool-calling-benchmark — github.com/MikeVeerman/tool-calling-benchmark
- [7] Patil et al., "The Berkeley Function Calling Leaderboard (BFCL)" — openreview.net/pdf?id=2GmDdhBdDk
- [8] "TinyLLM: Evaluation and Optimization of Small Language Models for Agentic Tasks on Edge Devices" — arxiv.org/pdf/2511.22138
- [9] "Best Local LLMs for Tool & Function Calling (2026 Tested)" — localaimaster.com/blog/best-ollama-models-tool-calling
- [10] "The Best Open-Source Small Language Models (SLMs) in 2026" — bentoml.com/blog/the-best-open-source-small-language-models
- [11] Bielik 11B v2 Technical Report (BFCL sub-task scores) — arxiv.org/pdf/2505.02410